

基于 MindSpore 的花卉识别实 验



华为技术有限公司

版权所有 © 华为技术有限公司 2024。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

华为技术有限公司

地址： 深圳市龙岗区坂田华为总部办公楼 邮编：518129

网址： <http://e.huawei.com>

目录

- 1 实验介绍.....1
 - 1.1 实验目的 1
 - 1.2 实验清单 1
 - 1.3 实验开发环境 1
- 2 花卉识别实验.....2
 - 2.1 花卉识别实验介绍 2
 - 2.2 实验总体设计 2
 - 2.2.1 功能结构 3
 - 2.2.2 体系结构 3
 - 2.3 实验详细设计与实现 4
 - 2.3.1 导入实验环境 4
 - 2.3.2 数据获取和处理 5
 - 2.3.3 构建花卉识别模型 7
 - 2.3.4 模型训练、测试 12
 - 2.3.5 花卉图像识别模型预测 14
 - 2.4 实验总结 15
- 3 附录：ModelArts 开发环境搭建 16

1 实验介绍

深度学习(DL, Deep Learning)是机器学习(ML, Machine Learning)领域中一个新的研究方向，它被引入机器学习使其更接近于最初的目标——人工智能(AI, Artificial Intelligence)。本章开设的实验主要涉及深度学习中的卷积神经网络。本章实验难度分为初级。

初级实验：花卉识别实验；

1.1 实验目的

本章实验的主要目的是了解并掌握深度学习卷积神经网络相关基础知识，在此基础上，基于华为自研 MindSpore 深度学习框架构建网络模型，实现图像识别(分类)相关任务。通过案例学习，最终学员能熟悉并掌握使用 MindSpore 深度学习框架实现深度学习实验任务。

1.2 实验清单

表格：实验、简述、难度、软件环境、硬件环境。

实验	简述	难度	软件环境	开发环境
花卉识别实验	基于MindSpore深度学习框架搭建卷积神经网络，实现花卉图片的分类。	中级	MindSpore2.1	ModelArts

1.3 实验开发环境

- Mindspore-2.1 (云端)

在华为云 ModelArts 上快速搭建开发环境，可参考文末附录：ModelArts 开发环境搭建。

2 花卉识别实验

2.1 花卉识别实验介绍

随着电子技术的迅速发展,人们使用便携数码设备(如手机、相机等)获取花卉图像越来越方便,如何自动识别花卉种类受到了广泛的关注。由于花卉所处背景的复杂性,以及花卉自身的类间相似性和类内多样性,利用传统的手工提取特征进行图像分类的方法,并不能很好地解决花卉图像分类这一问题。



图2-1 玫瑰图像

本实验基于华为自研的 MindSpore 深度学习开源框架,搭建卷积神经网络模型,实现花卉的识别。与传统图像分类方法不同,卷积神经网络无需人工提取特征,可以根据输入图像,自动学习包含丰富语义信息的特征,得到更为全面的花卉图像特征描述,可以很好地表达图像不同类别的信息。通过本实验的学习,相信大家对卷积神经网络会有深入的理解,对 MindSpore 深度学习框架的使用会有更深的掌握。

【实验环境要求】:

- 1、MindSpore2.1
- 3、ModelArts 平台

2.2 实验总体设计

该实验的设计可以从两个角度分析,一是功能结构,二是体系结构。

2.2.1 功能结构

花卉识别实验总体设计如图 2-2 所示，该实验可以划分为数据处理、模型构建、图像识别三个主要的子实验。其中数据处理子实验包括数据读取、数据集划分、图像预处理三部分；模型构建子实验主要包括模型定义、模型训练以及模型部署三个部分；图像识别子实验内容主要包括模型准备、模型应用两部分。

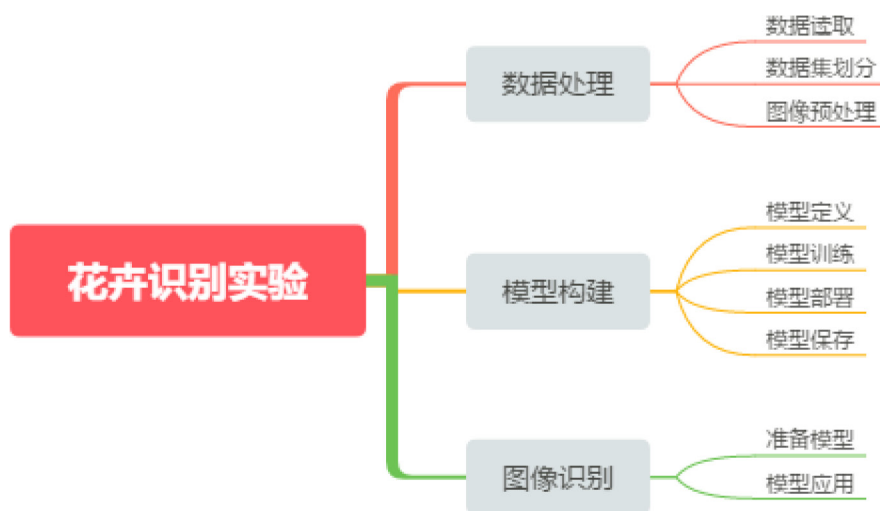


图2-2 实验整体功能结构图

2.2.2 体系结构

按照体系结构划分，整个实验的体系结构可以划分为三部分，分别为模型训练、模型保存和模型推理，如图 2-3 所示。各层侧重点各不相同。训练层运行在安装有 mindspore 框架的服务器，最好配置计算加速卡。推断层运行于开发环境，能够支持卷积神经网络的加速。展示层运行于客户端应用程序，能够完成图像选择并实时显示推断层的计算结果。各层之间存在单向依赖关系。推断层需要的网络模型由训练层提供，并根据需要进行必要的格式转换或加速重构。相应的，展示层要显示的元数据需要由推断层计算得到。

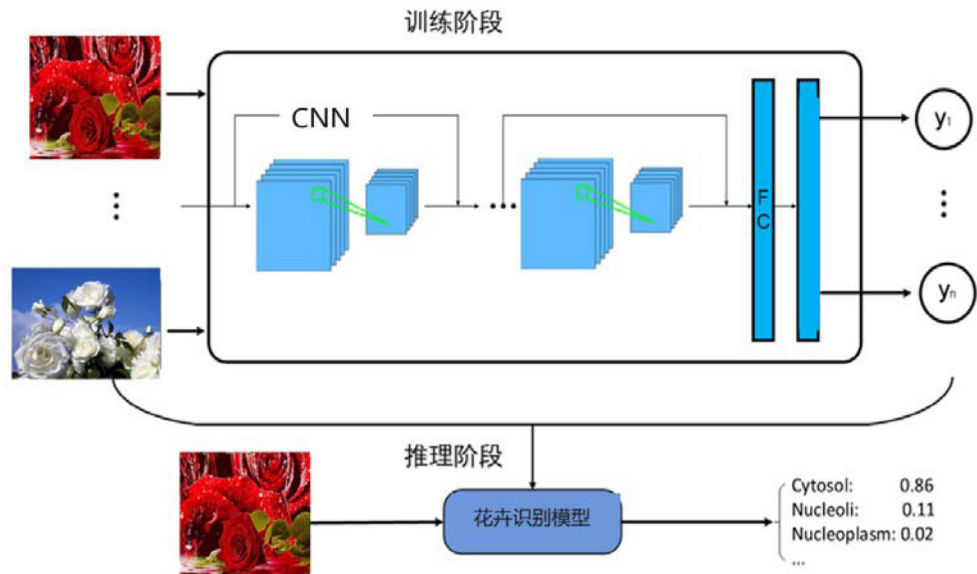


图2-3 实验流程图

2.3 实验详细设计与实现

本节将详细介绍实验的设计与实现。2.3.1 节导入实验环境；2.3.2 节数据读取和处理；2.3.3 节介绍构建花卉识别模型；2.3.4 节介绍如何保存训练后的模型；2.3.5 节则介绍如何进行模型的测试，以及相应测试结果的展示。

2.3.1 导入实验环境

步骤 1 导入相应的模块并初始化环境

glob 包主要用于查找符合特定规则的文件路径名，跟使用 windows 下的文件搜索相似；

os 模块主要用于处理文件和目录，比如：获取当前目录下文件，删除制定文件，改变目录，查看文件大小等。

```
#easydict 模块用于以属性的方式访问字典的值
from easydict import EasyDict as edict
#glob 模块主要用于查找符合特定规则的文件路径名，类似使用 windows 下的文件搜索
import glob
#os 模块主要用于处理文件和目录
import os
import numpy as np
import matplotlib.pyplot as plt

import mindspore as ms
#导入 mindspore 框架数据集
import mindspore.dataset as ds
```

```
#vision, transforms 模块是处理图像增强的高性能模块，用于数据增强图像数据改进训练模型。
from mindspore.dataset import vision, transforms
from mindspore.common import dtype as mstype

#导入模块用于初始化截断正态分布
from mindspore.common.initializer import TruncatedNormal
from mindspore import nn, Tensor, ops, load_checkpoint, load_param_into_net
from mindspore.train import ModelCheckpoint, CheckpointConfig, Model

# 设置 MindSpore 的执行模式和设备
ms.set_context(mode=ms.PYNATIVE_MODE, device_target="CPU") # "Ascend"、 "GPU" 、 "CPU"
```

步骤 2 定义超参数

```
cfg = edict({
    'data_path': 'flower_photos',
    'data_size': 3670,
    'image_width': 100,          # 图片宽度
    'image_height': 100,        # 图片高度
    'batch_size': 32,
    'channel': 3,                # 图片通道数
    'num_class': 5,              # 分类类别
    'weight_decay': 0.01,
    'lr': 0.0001,                # 学习率
    'dropout_ratio': 0.5,
    'epoch_size': 10,            # 训练次数
    'sigma': 0.01,

    'save_checkpoint_steps': 1,  # 多少步保存一次模型
    'keep_checkpoint_max': 1,    # 最多保存多少个模型
    'output_directory': './',   # 保存模型路径
    'output_prefix': "checkpoint_classification" # 保存模型文件名字
})
```

2.3.2 数据获取和处理

数据获取和处理流程如下：

- MindSpore 的 mindspore.dataset 提供了 ImageFolderDataset 函数，可以直接读取文件夹图片数据并映射文件夹名字为其标签(label)。这里我们使用 ImageFolderDataset 函数读取 'daisy','dandelion','roses','sunflowers','tulips'数据。并将这五类标签映射为：
{'daisy':0,'dandelion':1,'roses':2,'sunflowers':3,'tulips':4}
- 使用 RandomCropDecodeResize、HWC2CHW、TypeCast 进行数据预处理
- 按照 8:2 的比列将数据划分为训练数据集和测试数据集

- 对训练数据和测试数据分批次 (batch)

2.3.2.1 获取数据

这里已经准备好下载链接，运行以下代码就可以获取相应的数据集。

```
# 解压数据集，只需要第一次运行时解压，第二次无需再解压
%env no_proxy='a.test.com,127.0.0.1,2.2.2.2'
!wget https://ascend-professional-construction-dataset.obs.myhuaweicloud.com/deep-learning/flower_photos.zip
!unzip flower_photos.zip
```

2.3.2.2 读取数据集并可视化

```
#从目录中读取图像的源数据集。
de_dataset = ds.ImageFolderDataset(cfg.data_path,
                                   class_indexing={'daisy':0,'dandelion':1,'roses':2,'sunflowers':3,'tulips':4},
                                   shuffle=False)
trans_transform = [
    #解码前将输入图像裁剪成任意大小和宽高比。
    vision.RandomCropDecodeResize([cfg.image_width,cfg.image_height], scale=(0.08, 1.0), ratio=(0.75, 1.333)),
    #HWC2CHW：将张量格式从 NHWC 转换成 NCHW
    vision.HWC2CHW(),
]
#转换为给定 MindSpore 数据类型的 Tensor 操作。
type_cast_op = transforms.TypeCast(mstype.float32)

#将操作中的每个操作应用到此数据集。
de_dataset = de_dataset.map(input_columns="image", operations=trans_transform, num_parallel_workers=4)
de_dataset = de_dataset.map(input_columns="image", operations=type_cast_op, num_parallel_workers=4)

#划分训练集测试集
(de_train,de_test)=de_dataset.split([0.8,0.2], randomize=True)
#设置每个批处理的行数
#drop_remainder 确定是否删除最后一个可能不完整的批 ( default=False )。
#如果为 True，并且如果可用于生成最后一个批的 batch_size 行小于 batch_size 行，则这些行将被删除，并且
不会传播到子节点。
de_train = de_train.batch(cfg.batch_size, drop_remainder=True)
#重复此数据集计数次数。
de_test = de_test.batch(cfg.batch_size, drop_remainder=True)
print('训练数据集数量：',de_train.get_dataset_size()*cfg.batch_size)#get_dataset_size()获取批处理的大小。
print('测试数据集数量：',de_test.get_dataset_size()*cfg.batch_size)

#__next__方法处理后，获取一个 batch 的数据，数据格式为 NCHW，第一维度为 batch 的数量。
data_next = de_dataset.create_dict_iterator(output_numpy=True).__next__()
print('通道数/图像长/宽：', data_next['image'].shape)

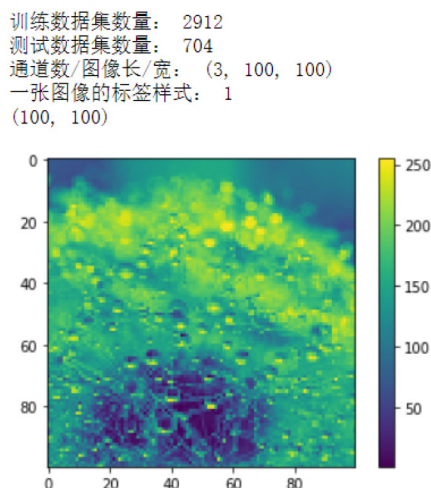
#一共 5 类，用 0-4 的数字表达类别。
```



```
print('一张图像的标签样式：', data_next['label'])
print(data_next['image'][0,...].shape)

plt.figure()
plt.imshow(data_next['image'][1,...])
plt.colorbar()
plt.grid(False)
plt.show()
```

输出结果：



2.3.3 构建花卉识别模型

2.3.3.1 网络结构框架介绍

花卉图像数据集准备完成，接下来我们就需要构建训练模型，本实验采用的是 CNN 神经网络算法，所以我们首先需要建立初始化的神经网络。

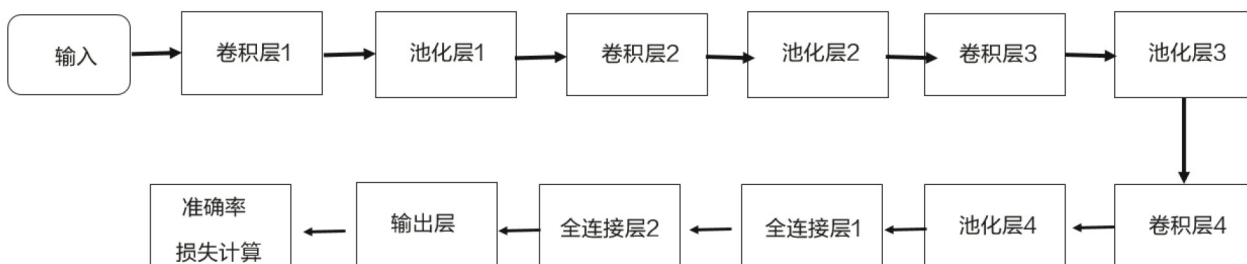


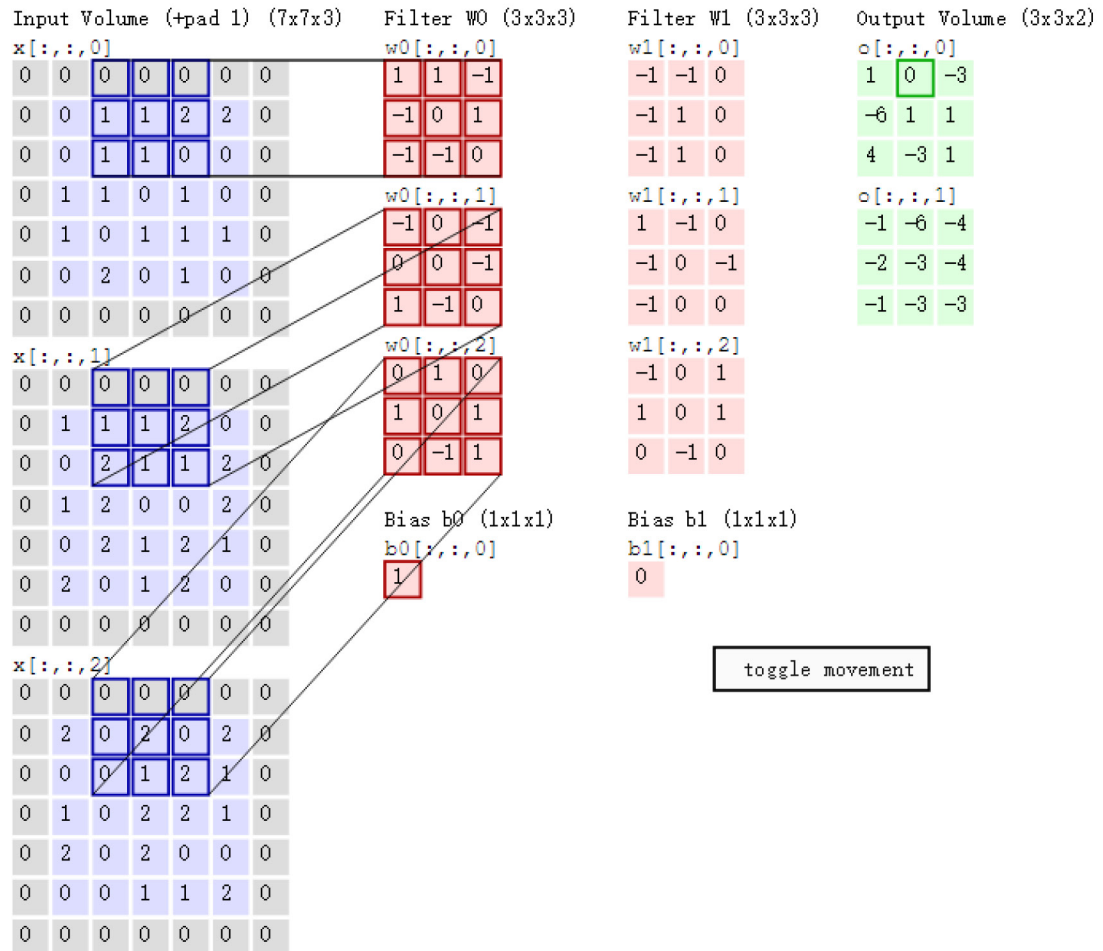
图2-4 神经网络框架图示意图

2.3.3.2 构建 CNN 神经网络

卷积层：

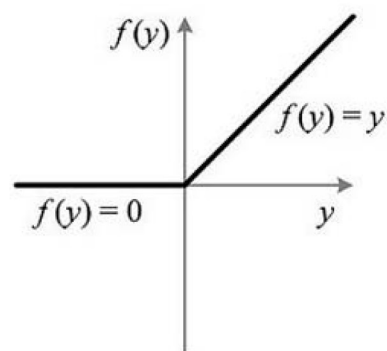
在卷积层计算过程中，输入是一定区域大小(width*height)的数据，和滤波器 filter（带着一组固定权重的神经元）做内积后得到新的二维数据。

具体来说，输入图像，然后就是滤波器 filter（带着一组固定权重的神经元）进行对应点乘，不同的滤波器 filter 会得到不同的输出数据，比如颜色深浅、轮廓。如果想提取图像的不同特征，则用不同的滤波器 filter，提取想要的关于图像的特定信息：颜色深浅或轮廓。



【更多资料】 <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>

激活函数：

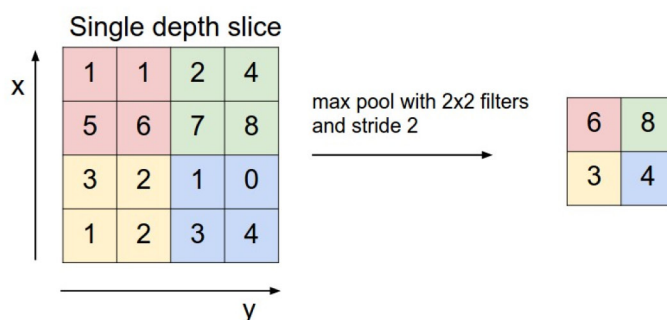


$$ReLU(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

ReLU 函数其实是分段线性函数，把所有的负值都变为 0，而正值不变，这种操作被称为单侧抑制。正因为有了这单侧抑制，才使得神经网络中的神经元具有了稀疏激活性。尤其体现在深度神经网络模型(如 CNN)中，当模型增加 N 层之后，理论上 ReLU 神经元的激活率将降低 2 的 N 次方倍。

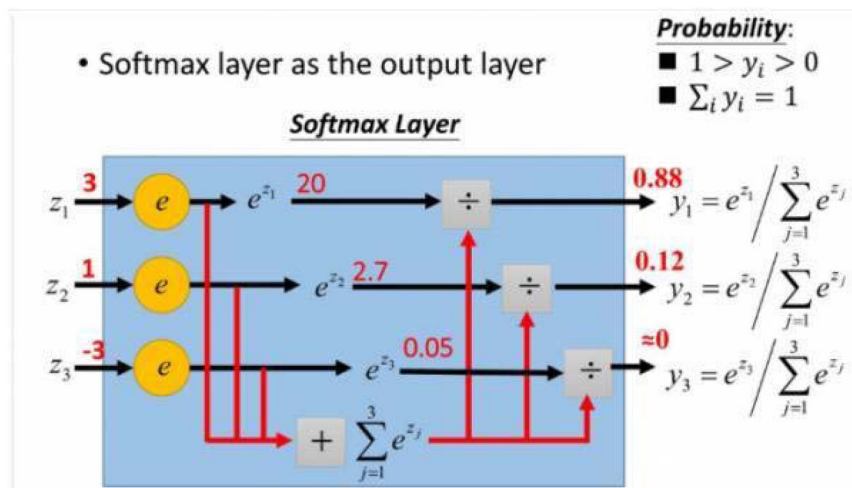
此外，相比于其它激活函数来说，ReLU 有以下优势：对于线性函数而言，ReLU 的表达能力更强，尤其体现在深度网络中；而对于非线性函数而言，ReLU 由于非负区间的梯度为常数 1，因此不存在梯度消失问题(Vanishing Gradient Problem)，使得模型的收敛速度维持在一个稳定状态。这里稍微描述一下什么是梯度消失问题：当梯度小于 1 时，预测值与真实值之间的误差每传播一层会衰减一次，如果在深层模型中使用 sigmoid 作为激活函数，这种现象尤为明显，将导致模型收敛停滞不前。

池化层：



上图所展示的是取区域最大，即上图左边部分中 左上角 2x2 的矩阵中 6 最大，右上角 2x2 的矩阵中 8 最大，左下角 2x2 的矩阵中 3 最大，右下角 2x2 的矩阵中 4 最大，所以得到上图右边部分的结果：6 8 3 4。

全连接层：



softmax 直白来说就是将原来输出是 3,1,-3 通过 softmax 函数作用，就映射成为(0,1)的值，而这些值的累和为 1（满足概率的性质），那么我们就可以将它理解成概率，在最后选取输出结点的时候，我们就可以选取概率最大（也就是值对应最大的）结点，作为我们的预测目标！

2.3.3.3 思考题

1. Softmax 函数是用于多分类任务的，如果是处理二分类任务我们用什么函数呢？
2. 请结合其他函数的特点，思考下 Relu 函数的优缺点？

【答案】

1、Sigmoid 函数

2、优点：

- （1）ReLU 的收敛速度比 sigmoid 和 tanh 快；（梯度不会饱和，解决了梯度消失问题）
- （2）计算复杂度低，不需要进行指数运算；
- （3）适合用于后向传播。

缺点：

- （1）ReLU 的输出不是 zero-centered；
- （2）Dead ReLU Problem（神经元坏死现象）：某些神经元可能永远不会被激活，导致相应参数永远不会被更新（在负数部分，梯度为 0）。产生这种现象的两个原因：参数初始化问题；learning rate 太高导致在训练过程中参数更新太大。解决方法：采用 Xavier 初始化方法，以及避免将 learning rate 设置太大或使用 adagrad 等自动调节 learning rate 的算法。
- （3）ReLU 不会对数据做幅度压缩，所以数据的幅度会随着模型层数的增加不断扩张。

2.3.3.4 定义网络

```
# 定义 CNN 图像识别网络
class Identification_Net(nn.Cell):
    def __init__(self, num_class=5, channel=3, dropout_ratio=0.5, trun_sigma=0.01): # 一共分五类，图片通道数是 3
        super(Identification_Net, self).__init__()
        self.num_class = num_class
        self.channel = channel
        self.dropout_ratio = dropout_ratio
        #设置卷积层
        self.conv1 = nn.Conv2d(self.channel, 32,
                                kernel_size=5, stride=1, padding=0,
                                has_bias=True, pad_mode="same",
                                weight_init=TruncatedNormal(sigma=trun_sigma), bias_init='zeros')

        #设置 ReLU 激活函数
        self.relu = nn.ReLU()
        #设置最大池化层
        self.max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2, pad_mode="valid")
        self.conv2 = nn.Conv2d(32, 64,
                                kernel_size=5, stride=1, padding=0,
                                has_bias=True, pad_mode="same",
                                weight_init=TruncatedNormal(sigma=trun_sigma), bias_init='zeros')

        self.conv3 = nn.Conv2d(64, 128,
                                kernel_size=3, stride=1, padding=0,
                                has_bias=True, pad_mode="same",
                                weight_init=TruncatedNormal(sigma=trun_sigma), bias_init='zeros')

        self.conv4 = nn.Conv2d(128, 128,
                                kernel_size=3, stride=1, padding=0,
                                has_bias=True, pad_mode="same",
                                weight_init=TruncatedNormal(sigma=trun_sigma), bias_init='zeros')

        self.flatten = nn.Flatten()
        self.fc1 = nn.Dense(6*6*128, 1024, weight_init=TruncatedNormal(sigma=trun_sigma), bias_init=0.1)
        self.dropout = nn.Dropout(self.dropout_ratio)
        self.fc2 = nn.Dense(1024, 512, weight_init=TruncatedNormal(sigma=trun_sigma), bias_init=0.1)
        self.fc3 = nn.Dense(512, self.num_class, weight_init=TruncatedNormal(sigma=trun_sigma), bias_init=0.1)

    #构建模型
    def construct(self, x):
        x = self.conv1(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.conv2(x)
        x = self.relu(x)
        x = self.max_pool2d(x)
        x = self.conv3(x)
        x = self.max_pool2d(x)
```

```
x = self.conv4(x)
x = self.max_pool2d(x)
x = self.flatten(x)
x = self.fc1(x)
x = self.relu(x)
x = self.dropout(x)
x = self.fc2(x)
x = self.relu(x)
x = self.dropout(x)
x = self.fc3(x)
return x
```

2.3.4 模型训练、测试

在进行正式的训练和测试之前，需要对模型进行实例化，定义优化器和损失函数，首先简单介绍损失函数及优化器的概念。

损失函数：又叫目标函数，用于衡量预测值与实际值差异的程度。深度学习通过不停地迭代来缩小损失函数的值。定义一个好的损失函数，可以有效提高模型的性能。

优化器：用于最小化损失函数，从而在训练过程中改进模型。

定义了损失函数后，可以得到损失函数关于权重的梯度。梯度用于指示优化器优化权重的方向，以提高模型性能。

MindSpore 支持的损失函数有 SoftmaxCrossEntropyWithLogits、L1Loss、MSELoss 等。这里使用 SoftmaxCrossEntropyWithLogits 损失函数。

MindSpore 提供了 callback 机制，可以在训练过程中执行自定义逻辑，这里使用框架提供的 ModelCheckpoint 为例。ModelCheckpoint 可以保存网络模型和参数，以便进行后续的 fine-tuning（微调）操作。

总共训练 10 个 epoch，每个 epoch 都需要在训练集上进行运行，并打印出相应的 loss 值，类似以下代码块的运行结果所示。如果运行在 GPU 或 Ascend 平台，epoch 可以设置得大一些，以提高模型得预测精度(实验证明：当在 Ascend/GPU 上训练，epoch_size 设置为 400 时，测试精度可以达到 90%左右)。另外，loss 值会波动，但总体来说 loss 值会逐步减小，精度逐步提高。每个人运行的 loss 值有一定随机性，不一定完全相同。

```
#实例化网络
net = Identification_Net(num_class=cfg.num_class, channel=cfg.channel, dropout_ratio=cfg.dropout_ratio)

#计算 softmax 交叉熵，以此作为损失函数
net_loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction="mean")
#设置 Adam 优化器，将前一步设定的需要更新的参数传入
net_opt = nn.Adam(net.trainable_params(), learning_rate=cfg.lr, weight_decay=0.0)

def train_loop(model, dataset, loss_fn, optimizer):
```

```
# 定义前向传播函数
def forward_fn(data, label):
    logits = model(data)
    loss = loss_fn(logits, label)
    return loss, logits

#获得梯度函数
grad_fn = ms.value_and_grad(forward_fn, None, optimizer.parameters, has_aux=True)

#定义单步训练函数
def train_step(data, label):
    (loss, _), grads = grad_fn(data, label)
    loss = ops.depend(loss, optimizer(grads))
    return loss

size = dataset.get_dataset_size()
model.set_train()
for batch, (data, label) in enumerate(dataset.create_tuple_iterator()):
    loss = train_step(data, label)

    if batch % 10 == 0:
        loss, current = loss.asnumpy(), batch
        print(f"loss: {loss:>7f}   [{current:>2d}/{size:>3d}]")

def test_loop(model, dataset, loss_fn):
    num_batches = dataset.get_dataset_size()
    model.set_train(False)
    total, test_loss, correct = 0, 0, 0
    for data, label in dataset.create_tuple_iterator():
        pred = model(data)
        total += len(data)
        test_loss += loss_fn(pred, label).asnumpy()
        correct += (pred.argmax(1) == label).asnumpy().sum()
    test_loss /= num_batches
    correct /= total
    print(f"Test: \n Accuracy: {(100*correct):>0.1f}%, Avg loss: {test_loss:>8f} \n")

epochs = cfg.epoch_size
curr_save_path = os.path.join(cfg.output_directory, f'{cfg.output_prefix}.ckpt')
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train_loop(net, de_train, net_loss, net_opt)
    ms.save_checkpoint(net, curr_save_path)
    test_loop(net, de_test, net_loss)
print("Done!")
```

2.3.4.1 思考题

1. Flatten 层用来将输入“压平”，即把多维的输入一维化，我们为什么要做这部分操作呢？

【答案】

- 1、将所有图片的像素值转化为一列，映射下一层的神经元。

2.3.5 花卉图像识别模型预测

2.3.5.1 模型预测

上一步的实验中我们已经设定了模型参数的保存路径，这里就直接加载 CKPT 文件然后利用模型进行推理。

```
#加载模型
import os

#实例化网络
net = Identification_Net(num_class=cfg.num_class, channel=cfg.channel, dropout_ratio=cfg.dropout_ratio)
#将参数加载进网络
param_dict = ms.load_checkpoint(curr_save_path)
param_not_load = ms.load_param_into_net(net, param_dict)
#编译整个模型
model = Model(net)
class_names = {0:'daisy',1:'dandelion',2:'roses',3:'sunflowers',4:'tulips'}
#获取测试集的 batch
test_ = de_test.create_dict_iterator().__next__()
#转换为 tensor
test = Tensor(test_['image'], ms.float32)
#预测
predictions = model.predict(test)
predictions = predictions.asnumpy()
true_label = test_['label'].asnumpy()

#显示预测结果
for i in range(9):
    p_np = predictions[i, :]
    pre_label = np.argmax(p_np)
    print('第' + str(i) + '个 sample 预测结果：', class_names[pre_label], '真实结果：', class_names[true_label[i]])
```

模型验证结果：

```
第0个sample预测结果: sunflowers 真实结果: sunflowers
第1个sample预测结果: dandelion 真实结果: dandelion
第2个sample预测结果: dandelion 真实结果: dandelion
第3个sample预测结果: daisy 真实结果: daisy
第4个sample预测结果: dandelion 真实结果: dandelion
第5个sample预测结果: dandelion 真实结果: dandelion
第6个sample预测结果: tulips 真实结果: tulips
第7个sample预测结果: dandelion 真实结果: dandelion
第8个sample预测结果: roses 真实结果: roses
```

图2-5 推理结果

2.4 实验总结

本章提供了一个基于华为自研 MindSpore 框架的图像识别实验。该实验演示了如何使用 MindSpore 框架搭建卷积神经网络，完成图像识别任务。本章对实验做了详尽的剖析。阐明了整个实验功能、结构与流程是如何设计的，详细解释了如何解析数据、如何构建深度学习模型、如何保存模型等内容。

训练保存后的模型在多个类别图片下进行测试，结果表明模型具有较快的推断速度和较好的识别性能。读者可以在该实验实验的基础上开发更有针对性的应用实验。

3 附录：ModelArts 开发环境搭建

在华为云 ModelArts 平台上创建 AI 框架为 Mindspore-2.1，硬件环境为 Ascend+ARM 的开发环境。

步骤 1 进入华为云 ModelArts 控制台

在[华为云 ModelArts 主页](#)，点击“管理控制台”进入 ModelArts 的管理页面。



图3-1 华为云 ModelArts 主页

步骤 2 创建 Notebook 训练作业

控制台区域选择“乌兰察布一”，在左侧菜单栏中选择“开发环境”的“Notebook”，点击进入 Notebook 创建页面。



图3-2 ModelArts 控制台

点击“创建”按钮，创建一个新的 Notebook，其配置如下：

- 名称：自定义。
- 自动停止：建议 1 小时。
- 镜像：mindspore_2.1.0，Ascend+ARM 算法开发和训练基础镜像。
- 资源池：公共资源池。
- 类型：ASCEND。
- 规格：Ascend: 1*Ascend|CPU: 24 核 96GB。

★ 自动停止 ☒

开启该选项后，该Notebook实例将在运行时长超出您所选择的时长后，自动停止。

☒ 1 小时 ☐ 2 小时 ☐ 4 小时 ☐ 6 小时 ☐ 自定义

★ 镜像

公共镜像

自定义镜像

请输入镜像名称

名称	描述
☐ pytorch1.8-cuda10.2-cudnn7-ubuntu18.04	CPU、GPU通用算法开发和训练基础镜像，预置AI引擎PyTorch1.8
☒ mindspore_2.1.0-cann_6.3.2-py_3.7-euler_2.10.7-aarch64	Ascend+ARM algorithm development and training. MindSpore is preset i...
☐ mindspore1.7.0-cuda10.1-py3.7-ubuntu18.04	GPU general algorithm development and training, preconfigured with AI...
☐ mindspore1.7.0-py3.7-ubuntu18.04	CPU general algorithm development and training, preconfigured with AI...
☐ pytorch1.10-cuda10.2-cudnn7-ubuntu18.04	CPU and GPU general algorithm development and training, preconfigur...
☐ tensorflow2.1-cuda10.1-cudnn7-ubuntu18.04	CPU、GPU通用算法开发和训练基础镜像，预置AI引擎TensorFlow2.1
☐ conda3-cuda10.2-cudnn7-ubuntu18.04	Clean user customized base image include cuda10.2, conda
☐ tensorflow1.13-cuda10.0-cudnn7-ubuntu18.04	GPU算法开发和训练基础镜像，预置AI引擎TensorFlow1.13.1
☐ pytorch1.4-cuda10.1-cudnn7-ubuntu18.04	CPU、GPU通用算法开发和训练基础镜像，预置AI引擎PyTorch1.4
☐ conda3-ubuntu18.04	Clean user customized base image only include conda

10 总条数: 14 < 1 2 >

★ 资源类型

公共资源池

专属资源池

★ 类型

ASCEND

★ 规格

Ascend: 1*Ascend910|ARM: 24核 96GB

昇腾910(32GB显存)单卡规格，配搭ARM处理器，适合深度学习场景下的模型训练和推理

★ 存储配置

云硬盘EVS

?

云硬盘EVS作为持久化存储挂载在/home/ma-user/work目录下，该目录下的内容在实例停止后会被保留，存储支持在线按需扩容

★ 磁盘规格 - 5 + GB

磁盘规格默认为5GB，从Notebook实例创建成功起，直至删除成功，每GB按照规定费用收费。

SSH远程开发

☐

Notebook 创建配置

配置完成之后“立即创建”，规格确认无误之后“提交”。

步骤 3 启动 Notebook 进入开发环境

当上一步创建好 Notebook 状态显示为“运行中”时，在右侧操作中“打开”，即可进入在线编程页面。

名称	状态	镜像	规格	描述	创建时间	创建者	操作
notebook-bc70	运行中 (1s)	tensorflow1.15-minds...	Ascend: 1*Ascend910(CPU: 24核 96...	CV1.7	2022/11/23 11:19:59 ...	yanxukun_bl...	打开 启动 停止 更多

可以在此页面创建或编辑 MindSpore 的项目，如图所示：

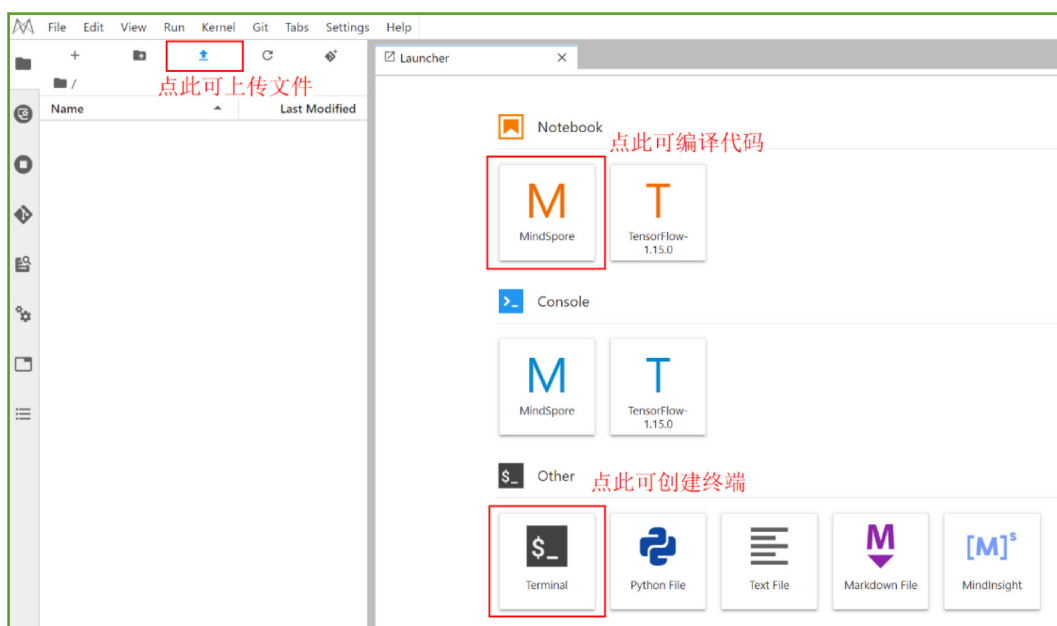


图3-3 Notebook 开发页面

*注意：Notebook 环境内上传、创建和编辑的文件均在/home/ma-user/work 目录下。

步骤 4 停止 Notebook 训练作业

实验完成之后，请及时关闭 Notebook 训练作业，避免产生不必要的资源浪费。

登录[华为云 ModelArts 控制台](#)，在“操作”栏选择“停止”操作。

如下图所示：



图3-4 及时停止 Notebook

至此训练用的线上 Notebook 环境搭建完成。